



COMPLETE SQL CHEATSHEET

Your Ultimate Quick Reference Guide



TABLE OF CONTENTS

1. Basic Queries
 2. Filtering & Conditions
 3. Sorting & Limiting
 4. Joins
 5. Aggregation & Grouping
 6. Subqueries
 7. Set Operations
 8. Data Modification
 9. Table Management
 10. Constraints
 11. Advanced Functions
 12. Window Functions
 13. CTEs (Common Table Expressions)
 14. Views
 15. Indexes
 16. Transactions
 17. Stored Procedures & Triggers
 18. Performance Tips
 19. Quick Syntax Reference
-

1 BASIC QUERIES

SELECT Statement

sql

-- Select all columns

```
SELECT * FROM table_name;
```

-- Select specific columns

```
SELECT column1, column2 FROM table_name;
```

-- Select with alias

```
SELECT column1 AS alias_name FROM table_name;
```

-- Select distinct values

```
SELECT DISTINCT column_name FROM table_name;
```

-- Select with calculation

```
SELECT column1, column2, column1 * column2 AS product FROM table_name;
```

LIMIT Clause

sql

-- Limit number of rows

```
SELECT * FROM table_name LIMIT 10;
```

-- Limit with offset

```
SELECT * FROM table_name LIMIT 10 OFFSET 20;
```

-- PostgreSQL specific

```
SELECT * FROM table_name FETCH FIRST 10 ROWS ONLY;
```

2 FILTERING & CONDITIONS

WHERE Clause

```
sql
-- Basic comparison
SELECT * FROM table_name WHERE column = 'value';
SELECT * FROM table_name WHERE column > 100;
SELECT * FROM table_name WHERE column BETWEEN 10 AND 20;

-- Text matching
SELECT * FROM table_name WHERE column LIKE 'A%'; -- Starts with A
SELECT * FROM table_name WHERE column LIKE '%A'; -- Ends with A
SELECT * FROM table_name WHERE column LIKE '%A%'; -- Contains A
SELECT * FROM table_name WHERE column LIKE 'A_ %'; -- Starts with A, at least 2 chars

-- IN operator
SELECT * FROM table_name WHERE column IN ('value1', 'value2');

-- NULL checks
SELECT * FROM table_name WHERE column IS NULL;
SELECT * FROM table_name WHERE column IS NOT NULL;

-- Multiple conditions
SELECT * FROM table_name
WHERE column1 = 'value'
  AND column2 > 100
  OR column3 IN ('A', 'B');
```

CASE Statement

```
sql
SELECT
  column,
  CASE
    WHEN condition1 THEN 'Result1'
    WHEN condition2 THEN 'Result2'
    ELSE 'Default'
  END AS new_column
FROM table_name;
```

3 SORTING & LIMITING

sql

-- Ascending order (default)

```
SELECT * FROM table_name ORDER BY column1;
```

-- Descending order

```
SELECT * FROM table_name ORDER BY column1 DESC;
```

-- Multiple columns

```
SELECT * FROM table_name ORDER BY column1 ASC, column2 DESC;
```

-- Random order

```
SELECT * FROM table_name ORDER BY RANDOM(); -- PostgreSQL
```

```
SELECT * FROM table_name ORDER BY RAND(); -- MySQL
```

JOINS

Types of Joins

sql

-- INNER JOIN (returns matching rows only)

SELECT t1.col1, t2.col2

FROM table1 t1

INNER JOIN table2 t2 ON t1.id = t2.id;

-- LEFT JOIN (all from left, matches from right)

SELECT t1.col1, t2.col2

FROM table1 t1

LEFT JOIN table2 t2 ON t1.id = t2.id;

-- RIGHT JOIN (all from right, matches from left)

SELECT t1.col1, t2.col2

FROM table1 t1

RIGHT JOIN table2 t2 ON t1.id = t2.id;

-- FULL OUTER JOIN (all from both)

SELECT t1.col1, t2.col2

FROM table1 t1

FULL OUTER JOIN table2 t2 ON t1.id = t2.id;

-- CROSS JOIN (Cartesian product)

SELECT * FROM table1 CROSS JOIN table2;

-- Self Join

SELECT a.name AS employee, b.name AS manager

FROM employees a

JOIN employees b ON a.manager_id = b.id;

-- Multiple Joins

SELECT *

FROM table1 t1

JOIN table2 t2 ON t1.id = t2.id

JOIN table3 t3 ON t2.id = t3.id;

5 AGGREGATION & GROUPING

Aggregate Functions

```
sql
COUNT(*)          -- Count all rows
COUNT(DISTINCT col) -- Count unique values
SUM(column)         -- Sum of values
AVG(column)         -- Average
MIN(column)         -- Minimum
MAX(column)         -- Maximum
STDDEV(column)      -- Standard deviation
VARIANCE(column)    -- Variance
```

GROUP BY

```
sql
SELECT
    department,
    COUNT(*) as count,
    AVG(salary) as avg_salary,
    MAX(salary) as max_salary
FROM employees
GROUP BY department;
```

```
-- Group by multiple columns
SELECT
    department,
    year,
    SUM(sales) as total_sales
FROM sales_data
GROUP BY department, year;
```

HAVING Clause (Filter groups)

```
sql
SELECT
    department,
    AVG(salary) as avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 50000;
```

```
-- HAVING with multiple conditions
SELECT
    department,
    COUNT(*) as count
FROM employees
WHERE hire_date > '2020-01-01'
GROUP BY department
HAVING COUNT(*) > 5 AND AVG(salary) > 60000;
```

SUBQUERIES

Types of Subqueries

```
sql
-- Subquery in WHERE (returns single value)
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- Subquery with IN (returns list)
SELECT * FROM employees
WHERE department_id IN (SELECT id FROM departments WHERE location = 'NYC');

-- Subquery with EXISTS
SELECT * FROM departments d
WHERE EXISTS (SELECT 1 FROM employees e WHERE e.department_id = d.id);

-- Subquery in SELECT
SELECT
    name,
    salary,
    (SELECT AVG(salary) FROM employees) AS avg_salary
FROM employees;

-- Subquery in FROM (derived table)
SELECT avg_salary
FROM (
    SELECT AVG(salary) as avg_salary
    FROM employees
    GROUP BY department
) AS dept_avgs;

-- Correlated subquery
SELECT * FROM employees e1
WHERE salary > (SELECT AVG(salary)
                FROM employees e2
                WHERE e2.department_id = e1.department_id);
```

7 SET OPERATIONS

sql

-- UNION (removes duplicates)

SELECT column1 FROM table1

UNION

SELECT column1 FROM table2;

-- UNION ALL (keeps duplicates)

SELECT column1 FROM table1

UNION ALL

SELECT column1 FROM table2;

-- INTERSECT (common rows)

SELECT column1 FROM table1

INTERSECT

SELECT column1 FROM table2;

-- EXCEPT (rows in first not in second)

SELECT column1 FROM table1

EXCEPT

SELECT column1 FROM table2;

-- MINUS (Oracle syntax)

SELECT column1 FROM table1

MINUS

SELECT column1 FROM table2;

8 DATA MODIFICATION

INSERT

sql

-- Insert single row

```
INSERT INTO table_name (col1, col2, col3)
```

```
VALUES ('value1', 123, 'value3');
```

-- Insert multiple rows

```
INSERT INTO table_name (col1, col2)
```

```
VALUES
```

```
    ('value1', 123),
```

```
    ('value2', 456),
```

```
    ('value3', 789);
```

-- Insert from SELECT

```
INSERT INTO table_name (col1, col2)
```

```
SELECT col1, col2 FROM another_table WHERE condition;
```

-- Insert with DEFAULT values

```
INSERT INTO table_name (id, name, created_at)
```

```
VALUES (DEFAULT, 'John', CURRENT_TIMESTAMP);
```

UPDATE

sql

-- Update single column

```
UPDATE table_name
```

```
SET column1 = 'new_value'
```

```
WHERE condition;
```

-- Update multiple columns

```
UPDATE table_name
```

```
SET column1 = 'new_value',
```

```
    column2 = 123
```

```
WHERE condition;
```

-- Update from another table

```
UPDATE table1 t1
```

```
SET t1.column1 = t2.column1
```

```
FROM table2 t2
```

```
WHERE t1.id = t2.id;
```

DELETE

sql

-- Delete specific rows

```
DELETE FROM table_name WHERE condition;
```

-- Delete all rows (faster than DELETE without WHERE)

```
TRUNCATE TABLE table_name;
```

-- Delete with JOIN

```
DELETE t1
```

```
FROM table1 t1
```

```
JOIN table2 t2 ON t1.id = t2.id
```

```
WHERE t2.status = 'inactive';
```

MERGE / UPSERT

sql

-- PostgreSQL

```
INSERT INTO table_name (id, name, count)
```

```
VALUES (1, 'John', 10)
```

```
ON CONFLICT (id)
```

```
DO UPDATE SET name = EXCLUDED.name, count = table_name.count + EXCLUDED.count;
```

-- MySQL

```
INSERT INTO table_name (id, name, count)
```

```
VALUES (1, 'John', 10)
```

```
ON DUPLICATE KEY UPDATE
```

```
    name = VALUES(name),
```

```
    count = count + VALUES(count);
```

-- SQL Server / Oracle

```
MERGE INTO table_name t
```

```
USING (SELECT 1 as id, 'John' as name, 10 as count) s
```

```
ON t.id = s.id
```

```
WHEN MATCHED THEN UPDATE SET
```

```
    name = s.name,
```

```
    count = t.count + s.count
```

```
WHEN NOT MATCHED THEN INSERT (id, name, count)
```

```
    VALUES (s.id, s.name, s.count);
```

TABLE MANAGEMENT

CREATE TABLE

sql

```
CREATE TABLE table_name (  
    id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    age INT CHECK (age >= 0),  
    email VARCHAR(255) UNIQUE,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    department_id INT,  
    FOREIGN KEY (department_id) REFERENCES departments(id)  
);
```

-- Create table from SELECT

```
CREATE TABLE new_table AS  
SELECT * FROM old_table WHERE condition;
```

-- Create temporary table

```
CREATE TEMP TABLE temp_table AS  
SELECT * FROM table_name WHERE 1=0;
```

ALTER TABLE

sql

-- Add column

```
ALTER TABLE table_name ADD column_name datatype;
```

-- Drop column

```
ALTER TABLE table_name DROP COLUMN column_name;
```

-- Rename column

```
ALTER TABLE table_name RENAME COLUMN old_name TO new_name;
```

-- Change column datatype

```
ALTER TABLE table_name ALTER COLUMN column_name TYPE new_datatype;
```

-- Add constraint

```
ALTER TABLE table_name ADD CONSTRAINT constraint_name UNIQUE (column);
```

DROP TABLE

sql

-- Drop table

```
DROP TABLE table_name;
```

-- Drop table with CASCADE (removes dependencies)

```
DROP TABLE table_name CASCADE;
```

-- Drop only if exists

```
DROP TABLE IF EXISTS table_name;
```

10 CONSTRAINTS

sql

-- PRIMARY KEY

```
CREATE TABLE table_name (  
    id INT PRIMARY KEY,  
    -- or  
    CONSTRAINT pk_name PRIMARY KEY (id)  
);
```

-- FOREIGN KEY

```
CREATE TABLE table_name (  
    dept_id INT,  
    CONSTRAINT fk_name FOREIGN KEY (dept_id)  
        REFERENCES departments(id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

-- UNIQUE

```
CREATE TABLE table_name (  
    email VARCHAR(255) UNIQUE,  
    -- or  
    CONSTRAINT unique_email UNIQUE (email)  
);
```

-- CHECK

```
CREATE TABLE table_name (  
    age INT CHECK (age >= 0 AND age <= 150),  
    -- or  
    CONSTRAINT age_check CHECK (age >= 0)  
);
```

-- NOT NULL

```
CREATE TABLE table_name (  
    name VARCHAR(100) NOT NULL  
);
```

-- DEFAULT

```
CREATE TABLE table_name (  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    status VARCHAR(20) DEFAULT 'active'  
);
```

-- Composite constraints

```
CREATE TABLE table_name (  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    CONSTRAINT pk_name PRIMARY KEY (first_name, last_name),  
    CONSTRAINT unique_name UNIQUE (first_name, last_name)  
);
```

1 1 ADVANCED FUNCTIONS

String Functions

```
sql
CONCAT(str1, str2)          -- Concatenate
LENGTH(str)                -- String length
UPPER(str) / LOWER(str)     -- Case conversion
TRIM(str)                   -- Remove spaces
SUBSTRING(str, start, length) -- Extract substring
REPLACE(str, old, new)       -- Replace text
LEFT(str, n) / RIGHT(str, n) -- Extract from left/right
POSITION('sub' IN str)       -- Find position
REVERSE(str)                 -- Reverse string
SPLIT_PART(str, delimiter, part) -- Split string (PostgreSQL)
```

Date/Time Functions

```
sql
CURRENT_DATE                -- Today's date
CURRENT_TIMESTAMP           -- Current timestamp
NOW()                       -- Current timestamp
DATE_PART('year', date)     -- Extract part (PostgreSQL)
EXTRACT(YEAR FROM date)     -- Extract part (SQL Standard)
AGE(date1, date2)           -- Difference (PostgreSQL)
DATE_ADD(date, INTERVAL 1 DAY) -- Add interval (MySQL)
DATE_DIFF('day', date1, date2) -- Difference
TO_DATE('2024-01-01', 'YYYY-MM-DD') -- Convert string to date
DATE_TRUNC('month', date)    -- Truncate date (PostgreSQL)
```

Math Functions

```
sql
ABS(n)                      -- Absolute value
CEIL(n) / FLOOR(n)          -- Round up/down
ROUND(n, decimals)          -- Round to decimal places
POWER(n, exponent)          -- Raise to power
SQRT(n)                     -- Square root
MOD(n, divisor)             -- Remainder
RANDOM()                     -- Random number (PostgreSQL)
RAND()                      -- Random number (MySQL)
```

Type Conversion

```
sql
CAST(column AS datatype)    -- Cast to datatype
column::datatype             -- PostgreSQL shorthand
TO_NUMBER('123.45', '999.99') -- String to number
TO_CHAR(date, 'YYYY-MM-DD')  -- Date to string
```

1 2 WINDOW FUNCTIONS

```
sql
-- ROW_NUMBER() - Sequential number within partition
SELECT
    name,
    salary,
    ROW_NUMBER() OVER (ORDER BY salary DESC) as rank
FROM employees;

-- RANK() - With gaps for ties
SELECT
    name,
    salary,
    RANK() OVER (ORDER BY salary DESC) as rank
FROM employees;

-- DENSE_RANK() - No gaps for ties
SELECT
    name,
    salary,
    DENSE_RANK() OVER (ORDER BY salary DESC) as dense_rank
FROM employees;

-- NTILE() - Divide into buckets
SELECT
    name,
    salary,
    NTILE(4) OVER (ORDER BY salary DESC) as quartile
FROM employees;

-- LAG() / LEAD() - Previous/Next values
SELECT
    name,
    salary,
    LAG(salary, 1) OVER (ORDER BY salary) as prev_salary,
    LEAD(salary, 1) OVER (ORDER BY salary) as next_salary
FROM employees;

-- SUM/AVG with window
SELECT
    name,
    salary,
    SUM(salary) OVER (PARTITION BY department) as dept_total,
    AVG(salary) OVER (PARTITION BY department) as dept_avg,
    salary * 100.0 / SUM(salary) OVER (PARTITION BY department) as pct_of_dept
FROM employees;

-- Running total
SELECT
    date,
    sales,
    SUM(sales) OVER (ORDER BY date) as running_total
FROM daily_sales;
```

-- Custom window frame

```
SELECT
    date,
    sales,
    AVG(sales) OVER (ORDER BY date ROWS BETWEEN 2 PRECEDING AND 2 FOLLOWING) as
moving_avg
FROM daily_sales;
```

-- Partition with order

```
SELECT
    name,
    department,
    salary,
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) as dept_rank
FROM employees;
```

1 3 CTEs (COMMON TABLE EXPRESSIONS)

sql

-- Basic CTE

```
WITH cte_name AS (  
    SELECT column1, column2  
    FROM table_name  
    WHERE condition  
)  
SELECT * FROM cte_name;
```

-- Multiple CTEs

```
WITH  
cte1 AS (  
    SELECT * FROM table1 WHERE condition  
)  
cte2 AS (  
    SELECT * FROM table2 WHERE condition  
)  
SELECT cte1.*, cte2.*  
FROM cte1  
JOIN cte2 ON cte1.id = cte2.id;
```

-- Recursive CTE

```
WITH RECURSIVE cte AS (  
    -- Anchor member  
    SELECT id, name, manager_id, 1 as level  
    FROM employees  
    WHERE manager_id IS NULL  
  
    UNION ALL  
  
    -- Recursive member  
    SELECT e.id, e.name, e.manager_id, c.level + 1  
    FROM employees e  
    JOIN cte c ON e.manager_id = c.id  
)  
SELECT * FROM cte;
```

-- CTE for complex calculations

```
WITH dept_stats AS (  
    SELECT  
        department,  
        AVG(salary) as avg_salary,  
        COUNT(*) as count  
    FROM employees  
    GROUP BY department  
)  
SELECT *  
FROM employees e  
JOIN dept_stats d ON e.department = d.department  
WHERE e.salary > d.avg_salary;
```

VIEWS

sql

-- Create view

```
CREATE VIEW view_name AS  
SELECT column1, column2  
FROM table_name  
WHERE condition;
```

-- Create or replace view

```
CREATE OR REPLACE VIEW view_name AS  
SELECT column1, column2  
FROM table_name  
WHERE condition;
```

-- Materialized view (PostgreSQL)

```
CREATE MATERIALIZED VIEW mv_name AS  
SELECT column1, COUNT(*)  
FROM table_name  
GROUP BY column1;
```

-- Refresh materialized view

```
REFRESH MATERIALIZED VIEW mv_name;
```

-- Drop view

```
DROP VIEW view_name;  
DROP VIEW IF EXISTS view_name CASCADE;
```

1 5 INDEXES

sql

-- Create index

```
CREATE INDEX idx_name ON table_name (column1);
```

-- Create unique index

```
CREATE UNIQUE INDEX idx_name ON table_name (column1);
```

-- Composite index

```
CREATE INDEX idx_name ON table_name (column1, column2);
```

-- Partial index (PostgreSQL)

```
CREATE INDEX idx_name ON table_name (column1) WHERE column2 = 'value';
```

-- Drop index

```
DROP INDEX idx_name;
```

-- Show indexes

```
SHOW INDEX FROM table_name; -- MySQL
```

```
SELECT * FROM pg_indexes WHERE tablename = 'table_name'; -- PostgreSQL
```

1 6 TRANSACTIONS

sql

-- Start transaction

BEGIN TRANSACTION;

-- or

START TRANSACTION;

-- Operations

INSERT INTO table_name VALUES ('value');

UPDATE table_name SET column = 'new' WHERE condition;

DELETE FROM table_name WHERE condition;

-- Checkpoint

SAVEPOINT savepoint_name;

-- Rollback to savepoint

ROLLBACK TO SAVEPOINT savepoint_name;

-- Commit changes

COMMIT;

-- Rollback all changes

ROLLBACK;

-- Auto-commit (MySQL)

SET AUTOCOMMIT = 0; -- Disable

SET AUTOCOMMIT = 1; -- Enable

1 7 STORED PROCEDURES & TRIGGERS

Stored Procedures

-- Create procedure (PostgreSQL)

```
CREATE OR REPLACE PROCEDURE proc_name(param1 INT, param2 TEXT)
LANGUAGE plpgsql
AS $$
BEGIN
    -- SQL statements
    INSERT INTO table_name VALUES (param1, param2);
END;
$$;
```

-- Call procedure

```
CALL proc_name(123, 'value');
```

-- Create function (returns value)

```
CREATE OR REPLACE FUNCTION func_name(param1 INT)
RETURNS INT
LANGUAGE plpgsql
AS $$
DECLARE
    result INT;
BEGIN
    SELECT COUNT(*) INTO result FROM table_name WHERE id = param1;
    RETURN result;
END;
$$;
```

-- Call function

```
SELECT func_name(123);
```

Triggers

-- Create trigger function

```
CREATE OR REPLACE FUNCTION trigger_function()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$;
```

-- Create trigger

```
CREATE TRIGGER trigger_name
BEFORE UPDATE ON table_name
FOR EACH ROW
EXECUTE FUNCTION trigger_function();
```

-- Drop trigger

```
DROP TRIGGER trigger_name ON table_name;
```

1 8 PERFORMANCE TIPS

EXPLAIN Plans

sql

-- Basic explain

```
EXPLAIN SELECT * FROM table_name WHERE condition;
```

-- Detailed explain

```
EXPLAIN ANALYZE SELECT * FROM table_name WHERE condition;
```

-- Explain with format

```
EXPLAIN (FORMAT JSON) SELECT * FROM table_name;
```

Query Optimization

sql

-- Use EXISTS instead of IN for large datasets

```
SELECT * FROM table1 t1
```

```
WHERE EXISTS (SELECT 1 FROM table2 t2 WHERE t2.id = t1.id);
```

-- Use UNION ALL instead of UNION when duplicates not a concern

```
SELECT col FROM table1
```

```
UNION ALL
```

```
SELECT col FROM table2;
```

-- Avoid SELECT *

```
SELECT specific_columns FROM table_name;
```

-- Use appropriate indexes

-- Covered indexes for frequently queried columns

-- Avoid functions in WHERE clauses

-- Bad: WHERE DATE(created_at) = '2024-01-01'

-- Good: WHERE created_at >= '2024-01-01' AND created_at < '2024-01-02'

-- Use LIMIT when testing queries

```
SELECT * FROM table_name LIMIT 100;
```

-- Batch operations

```
INSERT INTO table_name (col1, col2) VALUES
```

```
(1, 'a'), (2, 'b'), (3, 'c')...;
```

QUICK SYNTAX REFERENCE

Common Patterns

-- Pagination

```
SELECT * FROM table_name  
ORDER BY id  
LIMIT 20 OFFSET 40;
```

-- Find duplicates

```
SELECT column1, COUNT(*)  
FROM table_name  
GROUP BY column1  
HAVING COUNT(*) > 1;
```

-- Delete duplicates

```
DELETE FROM table_name  
WHERE id NOT IN (  
    SELECT MIN(id)  
    FROM table_name  
    GROUP BY duplicate_column  
);
```

-- Top N per group

```
SELECT *  
FROM (  
    SELECT *,  
        ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) as rank  
    FROM employees  
    ) ranked  
WHERE rank <= 3;
```

-- Date range

```
SELECT * FROM table_name  
WHERE created_at BETWEEN '2024-01-01' AND '2024-12-31';
```

-- Conditional updates

```
UPDATE table_name  
SET status = CASE  
    WHEN amount > 1000 THEN 'high'  
    WHEN amount > 500 THEN 'medium'  
    ELSE 'low'  
END  
WHERE date = CURRENT_DATE;
```

-- Generate series (PostgreSQL)

```
SELECT generate_series(1, 10);
```

-- Generate date series (PostgreSQL)

```
SELECT generate_series(  
    '2024-01-01'::date,  
    '2024-01-31'::date,  
    '1 day'::interval  
);
```

-- JSON operations (PostgreSQL)

SELECT json_build_object('id', id, 'name', name) FROM table_name;

SELECT data->>'key' FROM table_name;

-- String aggregation

SELECT department, STRING_AGG(name, ', ' ORDER BY name) as employees

FROM employees

GROUP BY department;

COMMON PITFALLS TO AVOID

1. ❌ **NULL comparisons:** Use `IS NULL`, not `= NULL`
 2. ❌ **Ambiguous columns:** Always alias or qualify column names in joins
 3. ❌ **Missing GROUP BY:** Non-aggregated columns must be in GROUP BY
 4. ❌ **Inefficient queries:** Use EXPLAIN to understand execution plans
 5. ❌ **Nested subqueries:** Can often be replaced with joins or CTEs
 6. ❌ **Implicit type conversion:** Can prevent index usage
 7. ❌ **Forgetting indexes:** Monitor and create appropriate indexes
 8. ❌ **Transaction span:** Keep transactions short and focused
 9. ❌ **No error handling:** Always use transaction rollback on errors
 10. ❌ **Hard-coded values:** Use parameters or variables instead
-

BEST PRACTICES

1. ✓ Use meaningful table and column names
 2. ✓ Always define PRIMARY KEY
 3. ✓ Use appropriate data types
 4. ✓ Index foreign keys and frequently queried columns
 5. ✓ Use NOT NULL where possible
 6. ✓ Avoid SELECT * in production
 7. ✓ Use prepared statements for dynamic queries
 8. ✓ Regular maintenance (VACUUM, ANALYZE for PostgreSQL)
 9. ✓ Backup before destructive operations
 10. ✓ Write self-documenting code with comments
-

USEFUL RESOURCES

- PostgreSQL: <https://www.postgresql.org/docs/>
 - MySQL: <https://dev.mysql.com/doc/>
 - SQL Server: <https://docs.microsoft.com/sql/>
 - Oracle: <https://docs.oracle.com/database/>
 - SQLite: <https://sqlite.org/docs.html>
-

 **Pro Tip:** Bookmark this cheatsheet and keep it handy while writing SQL queries!

Last Updated: 2026 | Happy Querying! 🎉